

Introduction

This reflective journal documents my individual learning experience while developing and evaluating a Convolutional Neural Network (CNN) and a transfer learning model to classify images of puppies and bagels. The goal of this exercise was not only to build a working model, but also to understand the full machine learning pipeline, including dataset preparation, model training, validation, evaluation, and interpretation of results.

Throughout the process, I encountered several practical challenges related to dataset structure, validation handling, and performance limitations due to small data size. These challenges significantly deepened my understanding of how CNNs behave in real-world scenarios.

Dataset Preparation & Structure

The first major learning point was understanding how image datasets must be structured for Keras. Initially, my dataset contained all images in a single folder, which caused Keras to detect 0 images and 0 classes. This helped me realize that `flow_from_directory()` relies entirely on folder names as class labels.

I resolved this by restructuring the dataset into two class folders:

```
puppy-or-bagel/  
├── bagel/  
└── puppy/
```

Once this structure was corrected, the data generator successfully detected 16 images across 2 classes.

```
• Found 12 images belonging to 2 classes.  
Found 4 images belonging to 2 classes.  
  
📊 Class mapping: {'bagel': 0, 'puppy': 1}  
✅ Train images: 12  
✅ Val images: 4  
✅ Batches per epoch (train): 2  
✅ Batches per epoch (val): 1
```

Reflection:

This step taught me that many model “errors” are actually data organization issues, not algorithmic ones. Proper dataset structure is a prerequisite for any successful deep learning workflow.

Custom CNN Training Observations

I trained a custom CNN using a very small dataset with data augmentation. During training, I noticed that:

- Training accuracy fluctuated heavily between epochs (25%–75%)
- Validation accuracy remained fixed at 50%
- Validation loss hovered around 0.69, which corresponds to random guessing

```

Epoch 1/12
1/1 _____ 0s 5s/step - accuracy: 0.7500 - loss: 0.6714
Epoch 1: val_accuracy improved from -inf to 0.50000, saving model to best_custom_model.keras
1/1 _____ 6s 6s/step - accuracy: 0.7500 - loss: 0.6714 - val_accuracy: 0.5000 - val_loss: 0.9735
Epoch 2/12
1/1 _____ 0s 2s/step - accuracy: 0.3750 - loss: 1.0464
Epoch 2: val_accuracy did not improve from 0.50000
1/1 _____ 2s 2s/step - accuracy: 0.3750 - loss: 1.0464 - val_accuracy: 0.5000 - val_loss: 0.6889
Epoch 3/12
1/1 _____ 0s 35ms/step - accuracy: 0.5000 - loss: 0.7323
Epoch 3: val_accuracy did not improve from 0.50000
1/1 _____ 0s 84ms/step - accuracy: 0.5000 - loss: 0.7323 - val_accuracy: 0.5000 - val_loss: 0.6881
Epoch 4/12
1/1 _____ 0s 10ms/step - accuracy: 0.7500 - loss: 0.6487
Epoch 4: val_accuracy did not improve from 0.50000
1/1 _____ 0s 56ms/step - accuracy: 0.7500 - loss: 0.6487 - val_accuracy: 0.5000 - val_loss: 0.6939
Epoch 5/12
1/1 _____ 0s 29ms/step - accuracy: 0.7500 - loss: 0.6206
Epoch 5: val_accuracy did not improve from 0.50000
1/1 _____ 0s 68ms/step - accuracy: 0.7500 - loss: 0.6206 - val_accuracy: 0.5000 - val_loss: 0.7139
Epoch 6/12
1/1 _____ 0s 14ms/step - accuracy: 0.3750 - loss: 0.8401
Epoch 6: val_accuracy did not improve from 0.50000
1/1 _____ 0s 60ms/step - accuracy: 0.3750 - loss: 0.8401 - val_accuracy: 0.5000 - val_loss: 0.6999
Epoch 7/12
1/1 _____ 0s 27ms/step - accuracy: 0.7500 - loss: 0.6105
Epoch 7: val_accuracy did not improve from 0.50000
1/1 _____ 0s 66ms/step - accuracy: 0.7500 - loss: 0.6105 - val_accuracy: 0.5000 - val_loss: 0.6981
Epoch 8/12
1/1 _____ 0s 13ms/step - accuracy: 0.3750 - loss: 0.7472
Epoch 8: val_accuracy did not improve from 0.50000
1/1 _____ 0s 59ms/step - accuracy: 0.3750 - loss: 0.7472 - val_accuracy: 0.5000 - val_loss: 0.6904
✓ Training complete!

```

Reflection:

Initially, I expected accuracy to steadily improve, but instead I learned that CNNs require large datasets to generalize. With such a small dataset, the model could not extract stable visual features and instead showed high variance.

Validation & Evaluation Insights

A key conceptual lesson was understanding why validation metrics were missing at first. I realized that validation was not being performed because no validation dataset had been defined. Once I used `validation_split` correctly, validation metrics became available.

Final evaluation of the custom CNN showed:

- Test Accuracy: 50%
- Test Loss: ~0.69

Custom CNN Results:
Test Accuracy: 0.5000 (50.00%)
Test Loss: 0.6881

Reflection:

This reinforced the importance of validation in assessing generalization. A model can appear to learn during training, but validation metrics reveal whether learning is meaningful.

Transfer Learning & Fine-Tuning Results

I then applied transfer learning using ResNet50, which is pretrained on ImageNet. Despite being pretrained, the model again achieved 50% accuracy, even after fine-tuning.

```
🚀 Training ResNet50 with Transfer Learning...  
  
Epoch 1/8  
1/1 ██████████ 12s 12s/step - accuracy: 0.6250 - loss: 0.7396  
Epoch 2/8  
1/1 ██████████ 5s 5s/step - accuracy: 0.5000 - loss: 0.9247  
Epoch 3/8  
1/1 ██████████ 1s 822ms/step - accuracy: 0.3750 - loss: 0.6962  
Epoch 4/8  
1/1 ██████████ 1s 1s/step - accuracy: 1.0000 - loss: 0.2384  
Epoch 5/8  
1/1 ██████████ 1s 1s/step - accuracy: 0.6250 - loss: 0.6722  
Epoch 6/8  
1/1 ██████████ 1s 842ms/step - accuracy: 0.7500 - loss: 0.5348  
Epoch 7/8  
1/1 ██████████ 1s 832ms/step - accuracy: 0.3750 - loss: 0.7319  
Epoch 8/8  
1/1 ██████████ 1s 803ms/step - accuracy: 0.7500 - loss: 0.6493  
  
✅ Transfer learning complete!
```

ResNet50 Transfer Learning Results:

Test Accuracy: 0.5000 (50.00%)

Test Loss: 0.6891

```
Epoch 1/5
1/1 _____ 23s 23s/step - accuracy: 0.5000 - loss: 0.7513 - val_accuracy: 0.5000 - val_loss: 0.6856
Epoch 2/5
1/1 _____ 8s 8s/step - accuracy: 0.2500 - loss: 1.1886 - val_accuracy: 0.5000 - val_loss: 0.6867
Epoch 3/5
1/1 _____ 0s 167ms/step - accuracy: 0.7500 - loss: 0.5064 - val_accuracy: 0.5000 - val_loss: 0.689
Epoch 4/5
1/1 _____ 0s 127ms/step - accuracy: 0.2500 - loss: 0.8182 - val_accuracy: 0.5000 - val_loss: 0.690
Epoch 5/5
1/1 _____ 0s 139ms/step - accuracy: 0.2500 - loss: 0.6878 - val_accuracy: 0.5000 - val_loss: 0.689
1/1 _____ 0s 78ms/step - accuracy: 0.5000 - loss: 0.6897
Fine-tuned Accuracy: 50.00%
```

Reflection:

This was one of the most important lessons of the lab. I learned that:

- Pretrained models are not “ready-made solutions”
- The classifier head still requires sufficient task-specific data
- Fine-tuning with extremely small datasets introduces noisy gradients rather than improvement

Transfer learning helps only when there is enough data to adapt pretrained features.

Model Behavior & Visual Confusion

By examining predictions, I observed that puppies and bagels can appear visually similar due to:

- Rounded shapes
- Brown/tan color similarities
- Zoomed or cropped images removing context
- Low resolution masking texture differences

Reflection:

This helped me understand how CNNs rely on visual cues rather than semantic understanding, and how limited data can cause the model to confuse shape and texture across classes.

Overall Reflection & Learning Outcomes

This lab shifted my understanding of deep learning from a “plug-and-play” mindset to a data-centric mindset. I learned that:

- Model performance is dominated by data quality and quantity
- Validation must be explicitly designed, not assumed
- Transfer learning is powerful but not magic
- Poor results can still represent a successful experiment if they are correctly interpreted

Most importantly, I learned how to diagnose model behavior, not just report metrics.

Conclusion

Although neither the custom CNN nor the transfer learning model achieved high accuracy, the experiment was successful in demonstrating the full CNN workflow and its limitations. The results highlight the critical role of dataset size and proper validation when training deep learning models.

This exercise strengthened my understanding of convolutional networks, transfer learning, and practical debugging in real machine learning pipelines.